ECEN 5113

HW: 2

Part A

Saira Khan

September 24, 2025

Procedure:

Vivado was used to create, synthesize, simulate, and test each design made in this section. An RTL project is created in Vivado, and design sources and simulation files are added to each project. A behavioral simulation is run to generate a waveform (timing schematic). The default FPGA part used is xc7k70tfbv67-1.

A Full Subtractor has three inputs: A (minuend), B (subtrahend), and Bin (Borrow-in from previous stage). It produces two outputs: D (difference) and Bout (indicates if a borrow is needed for the next stage). A Verilog module for the Subtractor was created based on these logic expressions:

$$D = (A \oplus B) \oplus Bin$$

$$Bout = A'Bin + A'B + BBin$$

The dataflow Verilog model was used to create this module. Assign statements defined the logic based on the above equations, and Boolean expressions (^, &, ~, |) were used to implement the operators. A testbench was created for simulation purposes, and different inputs of A and B were tried to test the logic based on the truth table for a 1-bit subtractor.

To create the 4-bit subtractor, the 1-bit module was used and instantiated four times, similar to the previously designed 4-bit ripple carry adder. Internal wires are declared that connect each 1-bit subtractor to the next. This allows for the borrow out of the previous subtractor to be the borrow in of the next subtractor. A testbench was created to test the logic of this module as well. Inputs A and B were changed along with Bin to test the borrow function of the subtractor.

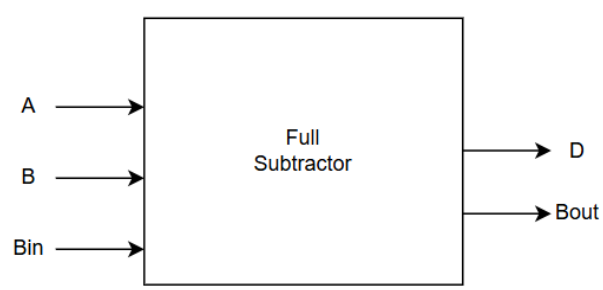

Figure 1: Block Diagram of Subtractor

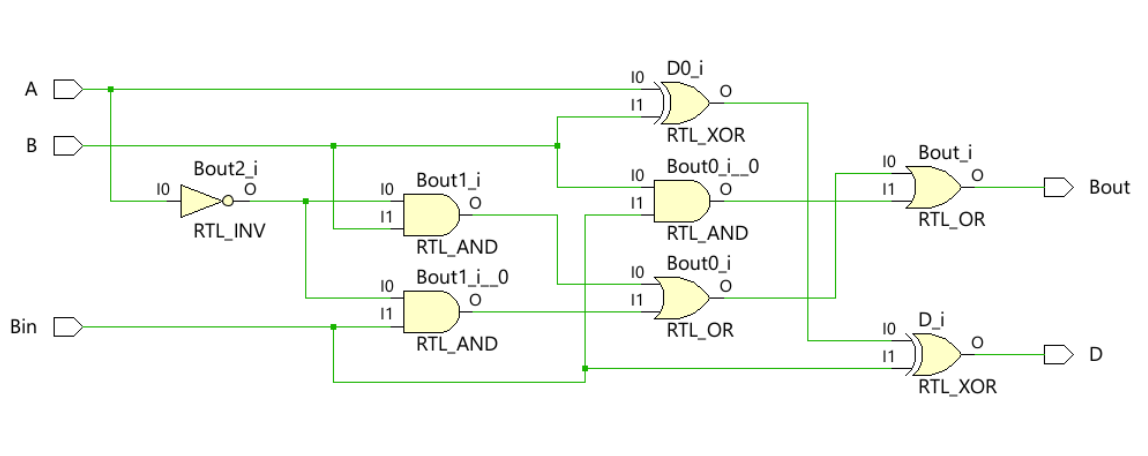

Figure 2:Circuit Diagram of 1-bit Subtractor

Table 1: Truth Table for 1-bit Subtractor

A	B	Bin	D	Bout
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

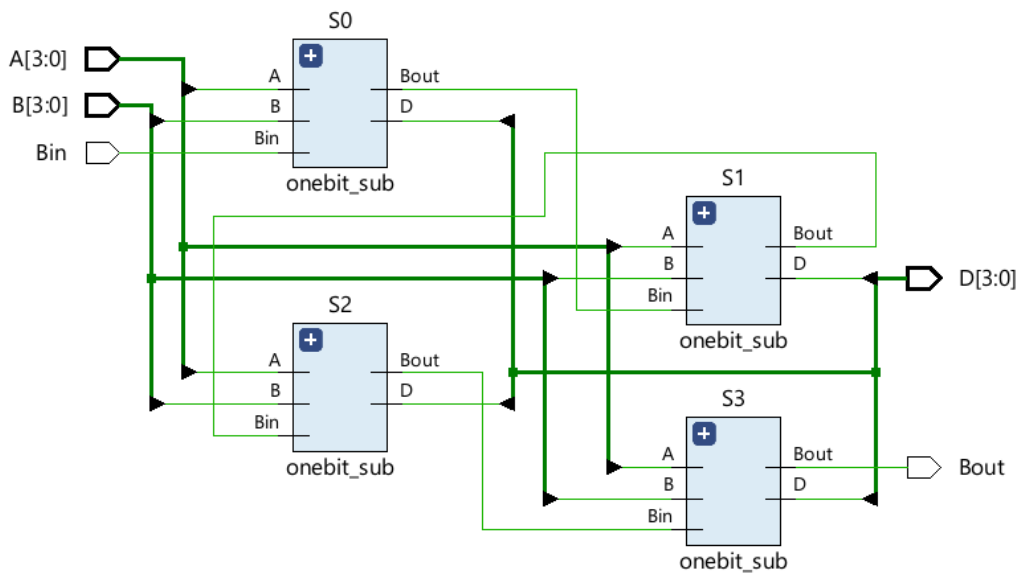

Figure 3:Circuit Diagram of 4-bit Subtractor

Observations:

One thing I noticed when building the module for the 4-bit subtractor was that it was very similar to building the 4-bit ripple carry adder. The Sum and Difference logical expressions are the same; the only difference occurs with the carry out/borrow out equations.

$$D = (A \oplus B) \oplus Bin$$

$$S = (A \oplus B) \oplus Cin$$

Building both modules involved first creating a module that had logic for a 1-bit adder/subtractor, then instantiating that module four times to create a 4-bit adder/subtractor. Instantiating the subtractor module means that the definition of the subtractor can be used with different inputs while the logic remains the same.

Results:

Figures 4 and 5 show the output of the 1-bit subtractor module synthesized/simulated in Vivado. Comparing Figure 4 to Table 1 shows that the subtractor module works correctly, as the results match the truth table. The waveform shows the change in input and the corresponding change in output for the testbench that is simulated.

```
# run 1000ns
time=10000 | A=0 | B=0 | Bin=0 | D=0 | Bout=0
time=20000 | A=0 | B=0 | Bin=1 | D=1 | Bout=1
time=30000 | A=0 | B=1 | Bin=0 | D=1 | Bout=1
time=40000 | A=0 | B=1 | Bin=1 | D=0 | Bout=1
time=50000 | A=1 | B=0 | Bin=0 | D=1 | Bout=0
time=60000 | A=1 | B=0 | Bin=1 | D=0 | Bout=0
time=70000 | A=1 | B=1 | Bin=0 | D=0 | Bout=0
time=80000 | A=1 | B=1 | Bin=1 | D=1 | Bout=1
```

Figure 4: Output of 1-bit Subtractor Module

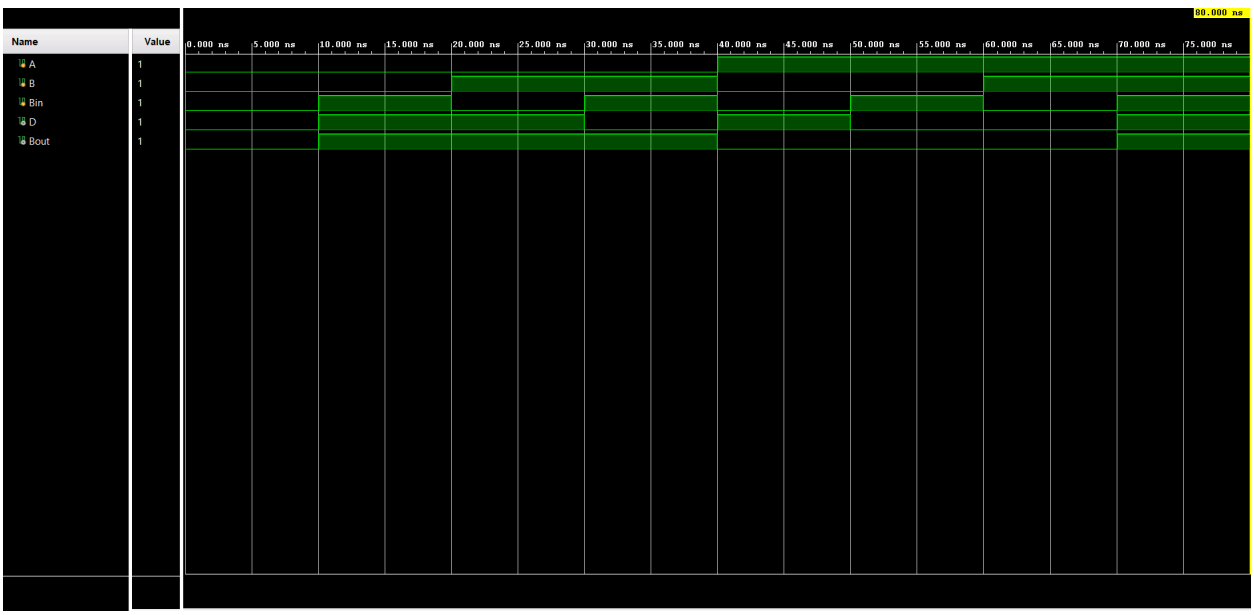

Figure 5: Waveform of 1-bit Subtractor

Looking at Figure 6, the outputs of the 4-bit module can be checked and verified manually. The first test shows that all the inputs are equal to 0, so the expected output is 0 with a Borrow-out of 0. The output simulated on Vivado confirms that the first test case has the right logic. The second test case shows that A=1001 (9), B=0011 (3), and there is a borrow-in of 0. The output, D, is 0110 (6), which is correct. The third test case shows that A=0101 (5), B=0110 (6), and Bin =0. The output shows to be 1111 (-1 in 2's complement) with a borrow out of 1. The last text case shows that A= 1111 (15), B=0001 (1), and there is a borrow of 1. The output shows the difference is equal to 1101 (13,) which is correct because it should be 15-1-1 = 13. The output simulated on Vivado then shows that the logic for the 4-bit subtractor works. The waveform shows the change in input and the corresponding output as the simulation runs. The x-axis shows the change in time, while the y-axis shows the different variables. A green area shows when there is a 1, and a black area shows a 0.

```
# run 1000ns
time=10000 | A=0000 | B=0000 | Bin=0 | D=0000 | Bout=0
time=20000 | A=1001 | B=0011 | Bin=0 | D=0110 | Bout=0
time=30000 | A=0101 | B=0110 | Bin=0 | D=1111 | Bout=1
time=40000 | A=1111 | B=0001 | Bin=1 | D=1101 | Bout=0
```

Figure 6: Output of 4-bit Subtractor

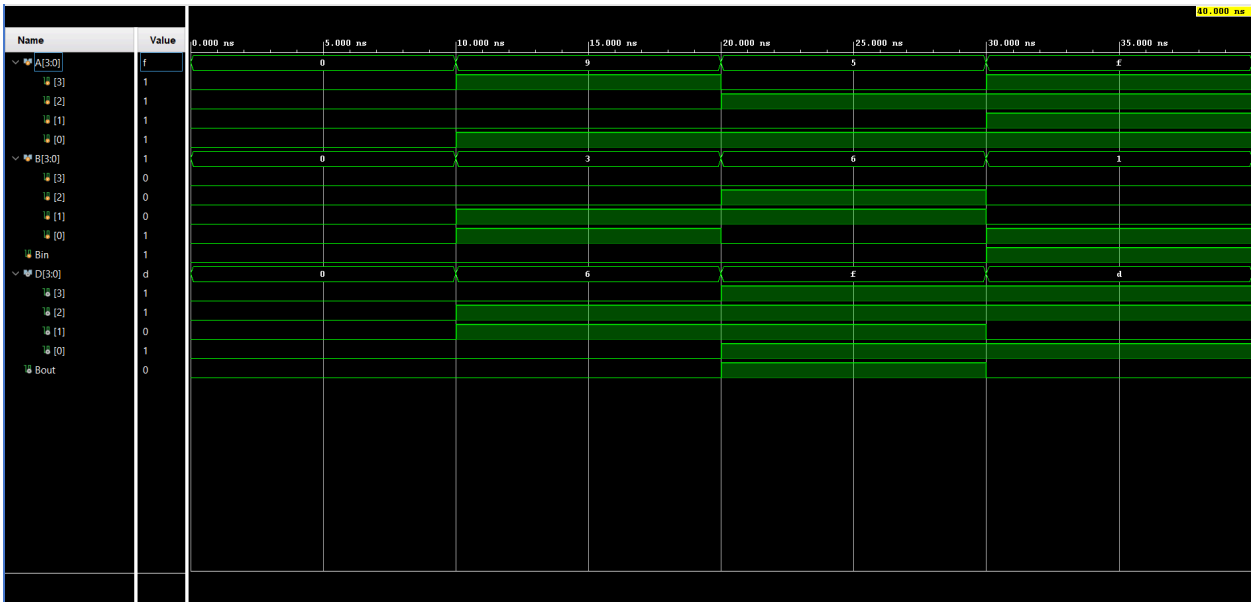

Figure 7: Waveform of a 4-bit Subtractor

Conclusion:

Part A of this lab is spent making a 1-bit subtractor and using that module to create a 4-bit subtractor by instantiating the 1-bit subtractor module. A comparison was drawn between the 4-bit ripple carry adder made in the last lab and the 4-bit subtractor created for this lab. The Dataflow Verilog model was used to create the modules in this lab because of its simplicity in allowing the modeling of connections between the subtractors. Structural modeling has a hierarchical design philosophy that allows designers to make complex structures, such as a 4-bit subtractor using a 1-bit subtractor module.

Appendix:

Verilog code for 1-bit Subtractor:

```
//define module
module onebit_sub(
        input A, //minuend
        input B, //subtrahend
        input Bin, //borrow in
        output D, //difference
        output Bout //borrow out
);
        //logic expression for difference
        assign D = (A ^ B) ^ Bin;
        //logic expression for Bout
        assign Bout = (~A & B) | (~A & Bin) | (B & Bin);
endmodule
```

Testbench code for 1-bit Subtractor:

```
//testbench
//define module
module onebit_sub_tb;
        //declare input/output
        reg A;
        reg B;
        reg Bin;
        wire D; //output
        wire Bout; //output
```

```
//instantiate module
onebit_sub d(A, B, Bin, D, Bout);
//initial block for setup
initial
        begin
                //initalize inputs
                A = 0;
                B = 0;
                Bin = 0;
                //test possible inputs combinations based on truth table
                #10; //add delay
                $display("time=%0t | A=%0b | B=%b | Bin=%b | D=%b | Bout=%b",
                                $time, A, B, Bin, D, Bout);
                A = 0; B=0; Bin= 1;
                #10;
                $display("time=%0t | A=%0b | B=%b | Bin=%b | D=%b | Bout=%b",
                                $time, A, B, Bin, D, Bout);
                A = 0; B = 1; Bin = 0;
                #10;
                $display("time=%0t | A=%0b | B=%b | Bin=%b | D=%b | Bout=%b",
                                $time, A, B, Bin, D, Bout);
                A = 0; B=1; Bin=1;
                #10;
                $display("time=%0t | A=%0b | B=%b | Bin=%b | D=%b | Bout=%b",
                                $time, A, B, Bin, D, Bout);
                A=1; B=0; Bin=0;
                #10;
                $display("time=%0t | A=%0b | B=%b | Bin=%b | D=%b | Bout=%b",
                                $time, A, B, Bin, D, Bout);
                A = 1; B=0; Bin=1;
```

```
                #10;
                $display("time=%0t | A=%0b | B=%b | Bin=%b | D=%b | Bout=%b",
                                $time, A, B, Bin, D, Bout);
                A=1; B=1; Bin=0;
                #10;
                $display("time=%0t | A=%0b | B=%b | Bin=%b | D=%b | Bout=%b",
                                $time, A, B, Bin, D, Bout);
                A = 1; B=1; Bin=1;
                #10;
                $display("time=%0t | A=%0b | B=%b | Bin=%b | D=%b | Bout=%b",
                                $time, A, B, Bin, D, Bout);
                $finish; //finish simulation
        end
    endmodule
```

Verilog Code for 4-bit Subtractor:

```
//define module
module onebit_sub(
    input A, //minuend
    input B, //subtrahend
    input Bin, //borrow in
    output D, //difference
    output Bout //borrow out



);

    //logic expression for difference
    assign D = (A ^ B) ^ Bin;

    //logic expression for Bout
```

```
        assign Bout = (~A & B) | (~A & Bin) | (B & Bin);

endmodule



//define module
module fourbit_sub (
        //define inputs/outputs
        input [3:0] A,
        input [3:0] B,
        input Bin,
        output [3:0] D,
        output Bout

);

        //internal wires used to connect the subtractors together
        wire c1, c2, c3;

        //instantiate four subtractors - similar to 4 bit ripple carry adder
        onebit_sub S0 (A[0], B[0], Bin, D[0], c1);
        onebit_sub S1 (A[1], B[1], c1, D[1], c2); //borrow out of S0 becomes borrow in for S1
        onebit_sub S2 (A[2], B[2], c2, D[2], c3);
        onebit_sub S3 (A[3], B[3], c3, D[3], Bout);

endmodule
```

Testbench for 4-bit Subtractor:

```
//testbench
//define module
module fourbit_sub_tb;
```

```
//declare inputs/outputs
reg [3:0] A;
reg [3:0] B;
reg Bin;
wire [3:0] D;
wire Bout;
//instantitate module
fourbit_sub d(A, B, Bin, D, Bout);
//iniital block for set up/simulating
initial
        begin
        //initialize inputs/outputs
        A = 4'b0000;
        B = 4'b0000;
        Bin = 1'b0;
        //testing different inputs
        #10;
        $display("time=%0t | A=%4b | B=%4b | Bin=%b | D=%4b | Bout=%b",
                                        $time, A, B, Bin, D, Bout);
        A = 4'b1001; B=4'b0011; Bin = 0;
        #10;
        $display("time=%0t | A=%4b | B=%4b | Bin=%b | D=%4b | Bout=%b",
                                        $time, A, B, Bin, D, Bout);
        A = 4'b0101; B=4'b0110; Bin = 0;
        #10;
        $display("time=%0t | A=%4b | B=%4b | Bin=%b | D=%4b | Bout=%b",
                                        $time, A, B, Bin, D, Bout);
        A = 4'b1111; B=4'b0001; Bin = 1;
        #10;
        $display("time=%0t | A=%4b | B=%4b | Bin=%b | D=%4b | Bout=%b",
```

```
                                        $time, A, B, Bin, D, Bout);
            $finish; //finish simulation
        end
endmodule
```